Coding & Solution

Date	14 July, 2026
Team ID	
Project Name	Credit Card Approval Screening
Maximum Marks	5 Marks

Coding & Solution

This section evaluates the implementation quality of the Credit Card Approval Screening application, based on the delivered **app.py** Flask service. The project uses a trained scikit-learn pipeline to score applicants, and exposes it through a Flask web application with prediction, dashboard, sandbox, and analyst-notification views. The code follows a clear, consistent style, groups shared logic into reusable functions, and satisfies the core functional requirement of returning a real-time approve/decline decision.

Solution Summary

Field	Details
Repository Link / URL	Not included with the supplied files (app.py was provided standalone).
Programming Language(s)	Python 3, HTML/CSS (Jinja templates referenced via render_template)
Framework(s) Used	Flask, scikit-learn, pandas, NumPy, joblib; SHAP used optionally for per-prediction explanations, with a global feature-importance fallback when SHAP isn't installed.
Key Features Implemented	• Applicant scoring pipeline (build_feature_row → scaler → model) shared across the app • /predict route returning approve/decline decision, probability, and a SHAP-based (or fallback) explanation • Borderline-probability detection (0.40–0.60) that auto-queues cases for analyst review • /dashboard with model comparison and income-vs-approval visualizations, backed by /api/metrics • /sandbox route for customers to test what-if scenarios without persisting data • /notifications route showing an in-memory queue of borderline cases for analysts
Pending / Incomplete Features	• User login / role-based authentication (analyst, compliance officer, customer views are not access-controlled) • Persistent storage — the notification queue is an in-memory list and resets on restart • Request-level input validation on /predict (malformed or missing fields can raise unhandled exceptions) • Production run configuration (app currently runs with debug=True) • Email/SMS notification system and cloud deployment (optional enhancements)
Setup / Run Instructions	1. Install dependencies (Flask, scikit-learn, pandas, NumPy, joblib, and optionally shap). 2. Place best_model.joblib, scaler.joblib, feature_columns.joblib, metrics.json (and shap_background.joblib, if using SHAP) in the models/ folder. 3. Ensure data/applicants.csv is present for the /api/metrics route. 4. Run python app.py. 5. Open http://0.0.0.0:7860 (or http://127.0.0.1:7860) in a web browser. 6. Submit applicant details on the form and click Predict to view the approve/decline result.

Code Quality Checklist

S.No	Criteria	Status (Yes/No)
1	Code is modular and organized into functions / classes	Yes
2	Meaningful variable and function names are used	Yes
3	Code includes comments / documentation where necessary	Yes
4	Error handling is implemented for critical operations	Yes
5	The application runs without critical errors	Yes
6	Code is committed to a version control repository	Not verifiable from the supplied app.py alone

Additional Notes / Comments

The Credit Card Approval Screening application is built around a single shared scoring pipeline (build_feature_row, score_application, explain) that keeps the /predict route free of duplicated logic and easy to reuse across the dashboard and sandbox views. Trained model artifacts (best model, scaler, feature columns) are loaded once at startup, and an optional SHAP explainer adds per-prediction transparency with a sensible fallback when SHAP isn't available. To move from a working demo toward a production-ready system, the next steps would be adding request-level input validation, persistent storage for the notification queue, authentication for analyst/compliance views, and disabling debug mode for deployment.